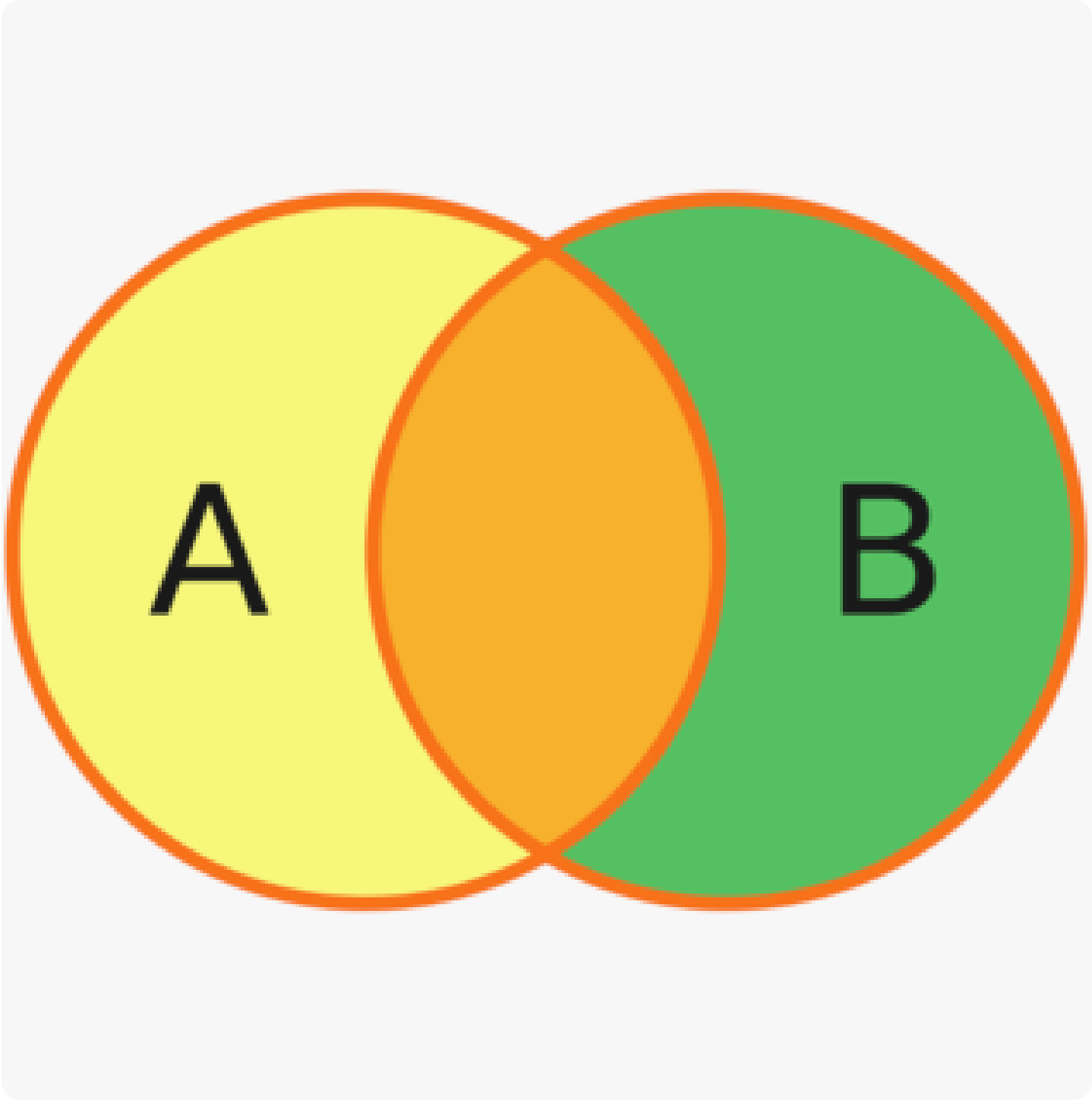




Join (SQL)

SQL clause
From Wikipedia, the free encyclopedia



A **join** clause in the Structured Query Language (SQL) combines columns from one or more tables into a new table. The operation corresponds to a join operation in relational algebra. Informally, a join stitches two tables and puts on the same row records with matching fields. There are several variants of `JOIN` : `INNER` , `LEFT OUTER` , `RIGHT OUTER` , `FULL OUTER` , `CROSS` , and others.

▾

Example tables

Summarize ✎

Perspective 🔄

To explain join types, the rest of this article uses the following tables:

More information

LastName, DepartmentID ... ▾

More information

DepartmentID, DepartmentName ... ▾

COVER IMAGE

A Venn diagram representing the full join SQL statement between tables A and B.

`Department.DepartmentID` is the primary key of the `Department` table, whereas `Employee.DepartmentID` is a foreign key.

Note that in `Employee`, "Williams" has not yet been assigned to a department. Also, no employees have been assigned to the "Marketing" department.

These are the SQL statements to create the above tables:

```
CREATE TABLE department(  
    DepartmentID INT PRIMARY KEY NOT NULL,  
    DepartmentName VARCHAR(20)  
);  
  
CREATE TABLE employee (  
    LastName VARCHAR(20),  
    DepartmentID INT REFERENCES department(DepartmentID)  
);  
  
INSERT INTO department  
VALUES (31, 'Sales'),  
       (33, 'Engineering'),  
       (34, 'Clerical'),  
       (35, 'Marketing');  
  
INSERT INTO employee  
VALUES ('Rafferty', 31),  
       ('Jones', 33),  
       ('Heisenberg', 33),  
       ('Robinson', 34),  
       ('Smith', 34),  
       ('Williams', NULL);
```

▼ Cross join

Summarize ↗

Perspective ↻

`CROSS JOIN` returns the Cartesian product of rows from tables in the join. In other words, it will produce rows which combine each row from the first table with each row from the second table.^[4]

More information Employee.LastName, Employee.DepartmentID ...



Example of an explicit cross join:

```
SELECT *  
FROM employee CROSS JOIN department;
```

Example of an implicit cross join:

```
SELECT *  
FROM employee, department;
```

The cross join can be replaced with an inner join with an always-true condition:

```
SELECT *  
FROM employee INNER JOIN department ON 1=1;
```

`CROSS JOIN` does not itself apply any predicate to filter rows from the joined table. The results of a `CROSS JOIN` can be filtered using a `WHERE` clause, which may then produce the equivalent of an inner join.

In the [SQL:2011](#) standard, cross joins are part of the optional F401, "Extended joined table", package.

Normal uses are for checking the server's performance.^{[[why?](#)]}

 1 Sources

Inner join

Summarize 

Perspective 

An **inner join** (or **join**) requires each row in the two joined tables to have matching column values, and is a commonly used join operation in [applications](#) but should not be assumed to be the best choice in all situations. Inner join creates a new result table by combining column values of two tables (A and B) based upon the join-predicate. The query compares each row of A with each row of B to find all pairs of rows that satisfy the join-predicate. When the join-predicate is satisfied by matching non-[NULL](#) values, column values for each matched pair of rows of A and B are combined into a result row.

The result of the join can be defined as the outcome of first taking the [cartesian product](#) (or [cross join](#)) of all rows in the tables (combining every row in table A with every row in table B) and then returning all rows that satisfy the join predicate. Actual SQL implementations normally use other approaches, such as [hash joins](#) or [sort-merge joins](#), since computing the Cartesian product is slower and would often require a prohibitively large amount of memory to store.

SQL specifies two different syntactical ways to express joins: the "explicit join notation" and the "implicit join notation". The "implicit join notation" is no longer considered a best practice^{[[by whom?](#)]}, although database systems still support it.

The "explicit join notation" uses the `JOIN` keyword, optionally preceded by the `INNER` keyword, to specify the table to join, and the `ON` keyword to specify the predicates for the join, as in the following example:

```
SELECT employee.LastName, employee.DepartmentID,  
       department.DepartmentName  
FROM employee  
INNER JOIN department ON  
       employee.DepartmentID = department.DepartmentID;
```

More information Employee.LastName, Employee.DepartmentID ...

The "implicit join notation" simply lists the tables for joining, in the `FROM` clause of the `SELECT` statement, using commas to separate them. Thus it specifies a [cross join](#), and the `WHERE` clause may apply additional filter-predicates (which function comparably to the join-predicates in the explicit notation).

The following example is equivalent to the previous one, but this time using implicit join notation:

```
SELECT employee.LastName, employee.DepartmentID,  
       department.DepartmentName  
FROM employee, department  
WHERE employee.DepartmentID = department.DepartmentID;
```

The queries given in the examples above will join the Employee and department tables using the DepartmentID column of both tables. Where the DepartmentID of these tables match (i.e. the join-predicate is satisfied), the query will combine the *LastName*, *DepartmentID* and *DepartmentName* columns from the two tables into a result row. Where the DepartmentID does not match, no result row is generated.

Thus the result of the execution of the query above will be:

More information Employee.LastName, Employee.DepartmentID ...

The employee "Williams" and the department "Marketing" do not appear in the query execution results. Neither of these has any matching rows in the other respective table: "Williams" has no associated department, and no employee has the department ID 35 ("Marketing"). Depending on the desired results, this behavior may be a subtle bug, which can be avoided by replacing the inner join with an outer join.

Inner join and NULL values

Programmers should take special care when joining tables on columns that can contain NULL values, since NULL will never match any other value (not even NULL itself), unless the join condition explicitly uses a combination predicate that first checks that the joins columns are **NOT NULL** before applying the remaining predicate condition(s). The Inner Join can only be safely used in a database that enforces referential integrity, or where the join columns are guaranteed not to be NULL. Many transaction processing relational databases rely on atomicity, consistency, isolation, durability (ACID) data update standards to ensure data integrity, making inner joins an appropriate choice. However, transaction databases usually also have desirable join columns that are allowed to be NULL. Many reporting relational database and data warehouses use high volume extract, transform, load (ETL) batch updates which make referential integrity difficult or impossible to enforce, resulting in potentially NULL join columns that an SQL query author cannot modify and which cause inner joins to omit data with no indication of an error. The choice to use an inner join depends on the database design and data characteristics. A left outer join can usually be substituted for an inner join when the join columns in one table may contain NULL values.

Any data column that may be NULL (empty) should never be used as a link in an inner join, unless the intended result is to eliminate the rows with the NULL value. If NULL join columns are to be deliberately removed from the result set, an inner join can be faster than an outer join because the table join and filtering is done in a single step. Conversely, an inner join can result in disastrously slow performance or even a server crash when used in a large volume query in combination with database functions in an SQL Where clause. ^[2] ^[3], ^[4] A function in an SQL Where clause can result in the database ignoring relatively compact table indexes. The database may read and inner join the selected columns from both tables before reducing the number of rows using the filter that depends on a calculated value, resulting in a relatively enormous amount of inefficient processing.

When a result set is produced by joining several tables, including master tables used to look up full-text descriptions of numeric identifier codes (a Lookup table), a NULL value in any one of the foreign keys can result in the entire row being eliminated from the result set, with no indication of error. A complex SQL query that includes one or more inner joins and several outer joins has the same risk for NULL values in the inner join link columns.

A commitment to SQL code containing inner joins assumes NULL join columns will not be introduced by future changes, including vendor updates, design changes and bulk processing outside of the application's data validation rules such as data conversions, migrations, bulk imports and merges.

One can further classify inner joins as equi-joins and theta (non-equi) joins.

Equi-join

The **equi-join**, also known as "the only eligible operation", is a specific type of comparator-based join, that uses only equality comparisons in the join-predicate. Using other comparison operators (such as `<`) disqualifies a join as an equi-join. The query shown above has already provided an example of an equi-join:

```
SELECT *
FROM employee JOIN department
ON employee.DepartmentID = department.DepartmentID;
```

We can write equi-join as below,

```
SELECT *
FROM employee, department
WHERE employee.DepartmentID = department.DepartmentID;
```

If columns in an equi-join have the same name, SQL-92 provides an optional shorthand notation for expressing equi-joins, by way of the `USING` construct:^[5]

```
SELECT *
FROM employee INNER JOIN department USING (DepartmentID);
```

The `USING` construct is more than mere syntactic sugar, however, since the result set differs from the result set of the version with the explicit predicate. Specifically, any columns mentioned in the `USING` list will appear only once, with an unqualified name, rather than once for each table in the join. In the case above, there will be a single `DepartmentID` column and no `employee.DepartmentID` or `department.DepartmentID`.

The `USING` clause is not supported by MS SQL Server and Sybase.

Natural join

The natural join is a special case of equi-join. Natural join ($\bowtie$) is a binary operator that is written as $(R \bowtie S)$ where R and S are relations.^[6] The result of the natural join is the set of all combinations of tuples in R and S that are equal on their common attribute names. For an example consider the tables *Employee* and *Dept* and their natural join:

More information $\bowtie$... 

This can also be used to define composition of relations. For example, the composition of *Employee* and *Dept* is their join as shown above, projected on all but the common attribute *DeptName*. In category theory, the join is precisely the fiber product.

The natural join is arguably one of the most important operators since it is the relational counterpart of logical AND. Note that if the same variable appears in each of two predicates that are connected by AND, then that variable stands for the same thing and both

appearances must always be substituted by the same value. In particular, the natural join allows the combination of relations that are associated by a foreign key. For example, in the above example a foreign key probably holds from *Employee.DeptName* to *Dept.DeptName* and then the natural join of *Employee* and *Dept* combines all employees with their departments. This works because the foreign key holds between attributes with the same name. If this is not the case such as in the foreign key from *Dept.manager* to *Employee.Name* then these columns have to be renamed before the natural join is taken. Such a join is sometimes also referred to as an **equi-join**.

More formally the semantics of the natural join are defined as follows:

$$R \bowtie S = \{t \cup s \mid t \in R \wedge s \in S \wedge \text{Fun}(t \cup s)\},$$

where *Fun* is a predicate that is true for a relation *r* if and only if *r* is a function. It is usually required that *R* and *S* must have at least one common attribute, but if this constraint is omitted, and *R* and *S* have no common attributes, then the natural join becomes exactly the Cartesian product.

The natural join can be simulated with Codd's primitives as follows. Let $c_1, \dots, c_m$ be the attribute names common to *R* and *S*, $r_1, \dots, r_n$ be the attribute names unique to *R* and let $s_1, \dots, s_k$ be the attributes unique to *S*. Furthermore, assume that the attribute names $x_1, \dots, x_m$ are neither in *R* nor in *S*. In a first step the common attribute names in *S* can now be renamed:

$$T = \rho_{x_1/c_1, \dots, x_m/c_m}(S) = \rho_{x_1/c_1}(\rho_{x_2/c_2}(\dots \rho_{x_m/c_m}(S) \dots))$$

Then we take the Cartesian product and select the tuples that are to be joined:

$$U = \pi_{r_1, \dots, r_n, c_1, \dots, c_m, s_1, \dots, s_k}(P)$$

A natural join is a type of equi-join where the **join** predicate arises implicitly by comparing all columns in both tables that have the same column-names in the joined tables. The resulting joined table contains only one column for each pair of equally named columns. In the case that no columns with the same names are found, the result is a cross join.

Most experts agree that NATURAL JOINS are dangerous and therefore strongly discourage their use.^[2] The danger comes from inadvertently adding a new column, named the same as another column in the other table. An existing natural join might then "naturally" use the new column for comparisons, making comparisons/matches using different criteria (from different columns) than before. Thus an existing query could produce different results, even though the data in the tables have not been changed, but only augmented. The use of column names to automatically determine table links is not an option in large databases with hundreds or thousands of tables where it would place an unrealistic constraint on naming conventions. Real world databases are commonly designed with foreign key data that is not consistently populated (NULL values are allowed), due to business rules and context. It is common practice to modify column names of similar data in different tables and this lack of rigid consistency relegates natural joins to a theoretical concept for discussion.

The above sample query for inner joins can be expressed as a natural join in the following way:

```
SELECT *
FROM employee NATURAL JOIN department;
```

As with the explicit `USING` clause, only one DepartmentID column occurs in the joined table, with no qualifier:

PostgreSQL, MySQL and Oracle support natural joins; Microsoft T-SQL and IBM DB2 do not. The columns used in the join are implicit so the join code does not show which columns are expected, and a change in column names may change the results. In the [SQL:2011](#) standard, natural joins are part of the optional F401, "Extended joined table", package.

In many database environments the column names are controlled by an outside vendor, not the query developer. A natural join assumes stability and consistency in column names which can change during vendor mandated version upgrades.

Theta (non-equi) join

 6 Sources

▼

Outer join

Summarize

🔍

Perspective

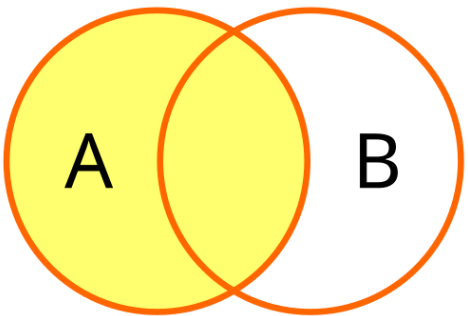
🔄

The joined table retains each row—even if no other matching row exists. Outer joins subdivide further into left outer joins, right outer joins, and full outer joins, depending on which table's rows are retained: left, right, or both (in this case *left* and *right* refer to the two sides of the `JOIN` keyword). Like [inner joins](#), one can further sub-categorize all types of outer joins as [equi-joins](#), [natural joins](#), `ON <predicate>` ([\$\theta\$ -join](#)), etc.^[8]

No implicit join-notation for outer joins exists in standard SQL.

Left outer join

The result of a **left outer join** (or simply **left join**) for tables A and B always contains all rows of the "left" table (A), even if the join-condition does not find any matching row in the "right" table (B). This means that if the `ON` clause matches 0 (zero) rows in B (for a given row in A), the join will still return a row in the result (for that row)—but with NULL in each column from B. A **left outer join** returns all the values from an inner join plus all values in the left table that do not match to the right table, including rows with NULL (empty) values in the link column.



A Venn diagram representing the left join SQL statement between tables A and B.

For example, this allows us to find an employee's department, but still shows employees that have not been assigned to a department (contrary to the inner-join example above, where unassigned employees were excluded from the result).

Example of a left outer join (the `OUTER` keyword is optional), with the additional result row (compared with the inner join) italicized:

```
SELECT *
FROM employee
LEFT OUTER JOIN department ON employee.DepartmentID =
department.DepartmentID;
```

Alternative syntaxes

Oracle supports the deprecated^[9] syntax:

```
SELECT *
FROM employee, department
WHERE employee.DepartmentID = department.DepartmentID(+)
```

Sybase supports the syntax (Microsoft SQL Server deprecated this syntax since version 2000):

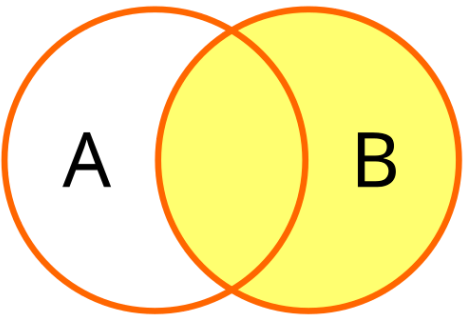
```
SELECT *
FROM employee, department
WHERE employee.DepartmentID *= department.DepartmentID
```

IBM Informix supports the syntax:

```
SELECT *
FROM employee, OUTER department
WHERE employee.DepartmentID = department.DepartmentID
```

Right outer join

A **right outer join** (or **right join**) closely resembles a left outer join, except with the treatment of the tables reversed. Every row from the "right" table (B) will appear in the joined table at least once. If no matching row from the "left" table (A) exists, NULL will appear in columns from A for those rows that have no match in B.



A Venn diagram representing the right join SQL statement between tables A and B.

A right outer join returns all the values from the right table and matched values from the left table (NULL in the case of no matching join predicate). For example, this allows us to find each employee and his or her department, but still show departments that have no employees.

Below is an example of a right outer join (the **OUTER** keyword is optional), with the additional result row italicized:

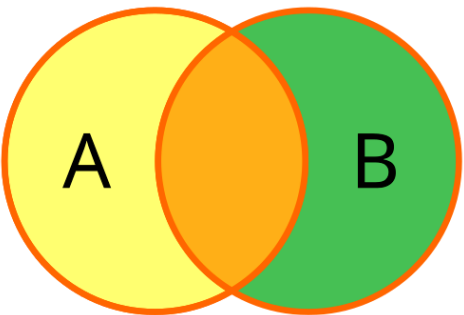
```
SELECT *
FROM employee RIGHT OUTER JOIN department
ON employee.DepartmentID = department.DepartmentID;
```

More information Employee.LastName, Employee.DepartmentID ...

Right and left outer joins are functionally equivalent. Neither provides any functionality that the other does not, so right and left outer joins may replace each other as long as the table order is switched.

Full outer join

Conceptually, a **full outer join** combines the effect of applying both left and right outer joins. Where rows in the full outer joined tables do not match, the result set will have NULL values for every column of the table that lacks a matching row. For those rows that do match, a single



row will be produced in the result set (containing columns populated from both tables).

A Venn diagram representing the full join SQL statement between tables A and B.

For example, this allows us to see each employee who is in a department and each department that has an employee, but also see each employee who is not part of a department and each department which doesn't have an employee.

Example of a full outer join (the **OUTER** keyword is optional):

```
SELECT *
FROM employee FULL OUTER JOIN department
ON employee.DepartmentID = department.DepartmentID;
```

More information Employee.LastName, Employee.DepartmentID ...

Some database systems do not support the full outer join functionality directly, but they can emulate it through the use of an inner join and UNION ALL selects of the "single table rows" from left and right tables respectively. The same example can appear as follows:

```
SELECT employee.LastName, employee.DepartmentID,
        department.DepartmentName, department.DepartmentID
FROM employee
INNER JOIN department ON employee.DepartmentID =
department.DepartmentID

UNION ALL

SELECT employee.LastName, employee.DepartmentID,
        cast(NULL as varchar(20)), cast(NULL as integer)
FROM employee
WHERE NOT EXISTS (
    SELECT * FROM department
    WHERE employee.DepartmentID =
department.DepartmentID)

UNION ALL

SELECT cast(NULL as varchar(20)), cast(NULL as integer),
        department.DepartmentName, department.DepartmentID
FROM department
WHERE NOT EXISTS (
    SELECT * FROM employee
    WHERE employee.DepartmentID =
department.DepartmentID)
```

Another approach could be UNION ALL of left outer join and right outer join MINUS inner join.

 2 Sources

Self-join

Summarize Perspective

A self-join is joining a table to itself.^[10]

Example

If there were two separate tables for employees and a query which requested employees in the first table having the same country as employees in the second table, a normal join operation could be used to find the answer table. However, all the employee information is contained within a single large table.^[11]

Consider a modified `Employee` table such as the following:

More information `EmployeeID, LastName ...`

An example solution query could be as follows:

```
SELECT F.EmployeeID, F.LastName, S.EmployeeID, S.LastName,
F.Country
FROM Employee F INNER JOIN Employee S ON F.Country = S.Country
WHERE F.EmployeeID < S.EmployeeID
ORDER BY F.EmployeeID, S.EmployeeID;
```

Which results in the following table being generated.

More information `EmployeeID, LastName ...`

For this example:

- `F` and `S` are aliases for the first and second copies of the employee table.
- The condition `F.Country = S.Country` excludes pairings between employees in different countries. The example question only wanted pairs of employees in the same country.
- The condition `F.EmployeeID < S.EmployeeID` excludes pairings where the `EmployeeID` of the first employee is greater than or equal to the `EmployeeID` of the second employee. In other words, the effect of this condition is to exclude duplicate pairings and self-pairings. Without it, the following less useful table would be generated (the table below displays only the "Germany" portion of the result):

More information `EmployeeID, LastName ...`

Only one of the two middle pairings is needed to satisfy the original question, and the topmost and bottommost are of no interest at all in this example.

W 2 Sources

Alternatives

The effect of an outer join can also be obtained using a UNION ALL between an INNER JOIN and a SELECT of the rows in the "main" table that do not fulfill the join condition. For example,

```
SELECT employee.LastName, employee.DepartmentID,
department.DepartmentName
FROM employee
LEFT OUTER JOIN department ON employee.DepartmentID =
department.DepartmentID;
```

Join (SQL)

Example tables

Cross join

Inner join

Inner join and NULL values

Equi-join

Natural join

Theta (non-equi) join

Outer join

Left outer join

Alternative syntaxes

Right outer join

Full outer join

Self-join

Example

Alternatives

Implementation

Join algorithms

Join indexes

Straight join

See also

References

Citations

Sources

Top Qs

Timeline

Chat

Perspective

can also be written as

```
SELECT employee.LastName, employee.DepartmentID,
department.DepartmentName
FROM employee
INNER JOIN department ON employee.DepartmentID =
department.DepartmentID

UNION ALL

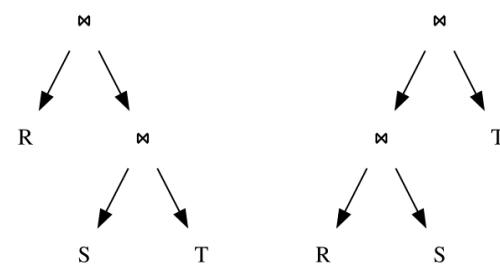
SELECT employee.LastName, employee.DepartmentID, cast(NULL as
varchar(20))
FROM employee
WHERE NOT EXISTS (
    SELECT * FROM department
    WHERE employee.DepartmentID =
department.DepartmentID)
```

Implementation

Summarize ✎

Perspective 🔄

Much work in database-systems has aimed at efficient implementation of joins, because relational systems commonly call for joins, yet face difficulties in optimising their efficient execution. The problem arises because inner joins operate both commutatively and associatively. In practice, this means that the user merely supplies the list of tables for joining and the join conditions to use, and the database system has the task of determining the most efficient way to perform the operation. The choices become more complex as the number of tables involved in a query increases, with each table having different characteristics in record count, average record length (considering NULL fields) and available indexes. Where Clause filters can also significantly impact query volume and cost.



Two possible query plans for the *triangle query* $R(A, B) \bowtie S(B, C) \bowtie T(A, C)$; the first joins S and T first and joins the result with R , the second joins R and S first and joins the result with T

A query optimizer determines how to execute a query containing joins. A query optimizer has two basic freedoms:

1. **Join order:** Because it joins functions commutatively and associatively, the order in which the system joins tables does not change the final result set of the query. However, join-order **could** have an enormous impact on the cost of the join operation, so choosing the best join order becomes very important.
2. **Join method:** Given two tables and a join condition, multiple algorithms can produce the result set of the join. Which algorithm runs most efficiently depends on the sizes of the input tables, the number of rows from each table that match the join condition, and the operations required by the rest of the query.

Many join-algorithms treat their inputs differently. One can refer to the inputs to a join as the "outer" and "inner" join operands, or "left" and "right", respectively. In the case of nested loops, for example, the database system will scan the entire inner relation for each row of the outer relation.

One can classify query-plans involving joins as follows:^[12]

left-deep

using a base table (rather than another join) as the inner operand of each join in the plan

right-deep

using a base table as the outer operand of each join in the plan

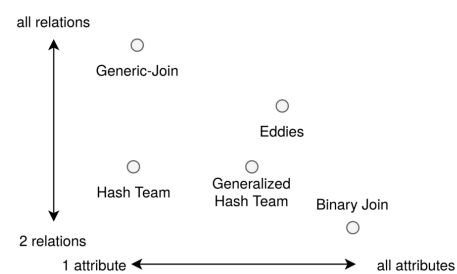
bushy

neither left-deep nor right-deep; both inputs to a join may themselves result from joins

These names derive from the appearance of the query plan if drawn as a tree, with the outer join relation on the left and the inner relation on the right (as convention dictates).

Join algorithms

Three fundamental algorithms for performing a binary join operation exist: nested loop join, sort-merge join and hash join. Worst-case optimal join algorithms are asymptotically faster than binary join algorithms for joins between more than two relations in the worst case.



An illustration of properties of join algorithms. When performing a join between more than two relations on more than two attributes, binary join algorithms such as hash join operate over two relations at a time, and join them on all attributes in the join condition; worst-case optimal algorithms such as generic join operate on a single attribute at a time but join all the relations on this attribute.^[13]

Join indexes

Join indexes are database indexes that facilitate the processing of join queries in data warehouses: they are currently (2012) available in implementations by Oracle^[14] and Teradata.^[15]

In the Teradata implementation, specified columns, aggregate functions on columns, or components of date columns from one or more tables are specified using a syntax similar to the definition of a database view: up to 64 columns/column expressions can be specified in a single join index. Optionally, a column that defines the primary key of the composite data may also be specified: on parallel hardware, the column values are used to partition the index's contents across multiple disks. When the source tables are updated interactively by users, the contents of the join index are automatically updated. Any query whose WHERE clause specifies any combination of columns or column expressions that are an exact subset of those defined in a join index (a so-called "covering query") will cause the join index, rather than the original tables and their indexes, to be consulted during query execution.

The Oracle implementation limits itself to using bitmap indexes. A *bitmap join index* is used for low-cardinality columns (i.e., columns containing fewer than 300 distinct values, according to the Oracle documentation): it combines low-cardinality columns from multiple related tables. The example Oracle uses is that of an inventory system, where different suppliers provide different parts. The schema has three linked tables: two "master tables", Part and Supplier, and a "detail table", Inventory. The last is a many-to-many table linking Supplier to Part, and contains the most rows. Every part has a Part Type, and every supplier is based in the US, and has a State column. There are not more than 60 states+territories in the US, and not more than 300 Part Types. The bitmap join index is defined using a standard three-table join on the three tables above, and specifying the Part_Type and Supplier_State columns for the index. However, it is defined on the Inventory table, even though the columns Part_Type and Supplier_State are "borrowed" from Supplier and Part respectively.

As for Teradata, an Oracle bitmap join index is only utilized to answer a query when the query's WHERE clause specifies columns limited to those that are included in the join index.

Straight join

Some database systems allow the user to force the system to read the tables in a join in a particular order. This is used when the join optimizer chooses to read the tables in an inefficient order. For example, in MySQL the command `STRAIGHT_JOIN` reads the tables in exactly the order listed in the query.^[16]

 5 Sources



See also

- [Join \(relational algebra\)](#).
- [Antijoin](#)
- [Set operations \(SQL\)](#).



References



External links



Edit in Wikipedia



Revision history



Read in Wikipedia



RELATED TOPICS

W

Relational model
Database model



Data definition language
Syntax for defining data structures

W

Surrogate key
Unique identifier for either an entity in the modeled world or an object in the database

W

Select (SQL)
SQL statement that returns a result set of rows from one or more tables

W

Insert (SQL)
SQL statement

Show more 



Wikiwand - on Chrome

Seamless Wikipedia browsing. On steroids.

